AULA VIRTUAL IFTS° 21

Sistema Web — Documentación Técnica

Sistema de Gestión Académica — Tecnicatura en Análisis de Sistemas

Versión	1.0.0
Fecha	2025
Tipo de Proyecto	Proyecto Académico — Enfocado en REST API
Clasificación	Uso Interno / Educativo

Tabla de Contenidos

Tabla de Contenidos 2
1. Introducción 4
1.1 Alcance del Documento 4
1.2 Repositorios 4
2. Visión General del Sistema 5
2.1 Módulos Funcionales 5
2.2 Roles de Usuario 5
3. Arquitectura del Sistema 7
3.1 Capas de la Aplicación 7
4. Modelo de Datos 8
4.1 Diagrama Entidad-Relación (DER) 8
4.2 Descripción de Entidades 8
4.3 Detalle de Entidades Principales 9
4.3.1 Entidad: user 9
4.3.2 Entidad: subject_inscribed 9
4.3.3 Entidad: final_exam_registered 10
4.3.4 Sistema RBAC: role / permission / role_permissions / user_roles 10
5. Stack Tecnológico 11
6. Guía de Instalación y Configuración 12
6.1 Requisitos del Entorno 12
6.2 Pasos de Instalación 12
Paso 1 — Clonar el Repositorio 12
Paso 2 — Configurar la Base de Datos 12
Paso 3 — Ejecutar el Proyecto 12
Paso 4 — Ejecución con Docker (Opcional) 13
7. Configuración Avanzada 14
7.1 Perfiles de Spring 14
7.2 Variables de Entorno Recomendadas 14
8. Consideraciones de Diseño y Mejores Prácticas 15
8.1 Seguridad 15
8.2 Calidad de Código 15
8.3 Escalabilidad Futura 15
9. Glosario 16

10. Historial de Versiones 17

1. Introducción

Este documento constituye la documentación técnica oficial del componente backend del sistema Aula Virtual, desarrollado en el marco de la Tecnicatura en Análisis de Sistemas. La presente especificación está destinada a desarrolladores, integradores y personal técnico que interactúen con la arquitectura del sistema.

El sistema provee una API RESTful que centraliza la lógica de negocio académica, incluyendo la gestión de usuarios, materias, historial académico, exámenes y el control de acceso basado en roles (RBAC).

1.1 Alcance del Documento

Este documento cubre los siguientes aspectos del sistema:

- Descripción general de la arquitectura y módulos funcionales
- Modelo de datos relacional (DER) y descripción de entidades
- Stack tecnológico utilizado
- Instrucciones de instalación y configuración del entorno
- Guía de configuración de la aplicación

1.2 Repositorios

Componente	URL
Backend (este repo)	https://github.com/tu_usuario/aula-virtual-backend
Frontend	https://github.com/nxhuel/aula-virtual-frontend

2. Visión General del Sistema

El Aula Virtual es un sistema de gestión académica diseñado para centralizar y digitalizar los procesos educativos de una institución de nivel terciario. La solución adopta una arquitectura cliente-servidor desacoplada, donde el backend expone una API REST consumida por el frontend y potencialmente por integraciones de terceros.

2.1 Módulos Funcionales

Módulo	Funcionalidades	Actores Involucrados
Gestión de Usuarios	CRUD completo, cambio de estado (habilitado/bloqueado), recupero de contraseña	Administrativo, Estudiante, Docente
Roles y Permisos	Asignación de roles, gestión de permisos por rol, control de acceso a endpoints	Administrativo
Programas Académicos	Alta/baja/modificación de carreras, carga del plan de estudios	Administrativo
Gestión de Materias	ABM de materias, asignación de docente, configuración de plan académico	Administrativo, Docente
Inscripción a Materias	Registro de cursada, carga de calificaciones parciales y finales	Estudiante, Docente
Exámenes Finales	Creación de mesas, inscripción de alumnos, registro de asistencia y nota	Estudiante, Docente, Administrativo
Historial Académico	Consulta de trayectoria, materias aprobadas, promedios, estado académico	Estudiante, Administrativo

2.2 Roles de Usuario

El sistema contempla tres tipos de actores principales, gestionados mediante el módulo de roles y permisos (RBAC):

Rol	Responsabilidades principales
ADMINISTRATIVO	Gestión integral: usuarios, carreras, materias, configuración del sistema
DOCENTE	Gestión de materias asignadas, carga de calificaciones, presencia en mesas examinadoras

ESTUDIANTE	Inscripción a materias y exámenes, consulta de historial académico y calificaciones

3. Arquitectura del Sistema

El backend sigue una arquitectura en capas (Layered Architecture) propia del ecosistema Spring Boot, con separación clara de responsabilidades entre las capas de presentación (controladores REST), lógica de negocio (servicios) y acceso a datos (repositorios JPA).

3.1 Capas de la Aplicación

Capa	Componente Spring	Responsabilidad
Presentación	`@RestController`	Exposición de endpoints REST, validación de entrada, serialización JSON
Seguridad	`Spring Security / JWT`	Autenticación, autorización, filtrado de peticiones, gestión de tokens
Negocio	`@Service`	Lógica de negocio, orquestación de operaciones, reglas académicas
Persistencia	`@Repository / JPA`	Acceso a datos, queries JPQL/HQL, transacciones de base de datos
Base de Datos	`MySQL 8.x`	Almacenamiento relacional, integridad referencial, índices

4. Modelo de Datos

El diseño de la base de datos sigue el paradigma relacional, con entidades normalizadas hasta la Tercera Forma Normal (3FN). A continuación se presenta el Diagrama Entidad-Relación (DER) completo del sistema, seguido de la descripción detallada de cada entidad.

4.1 Diagrama Entidad-Relación (DER)

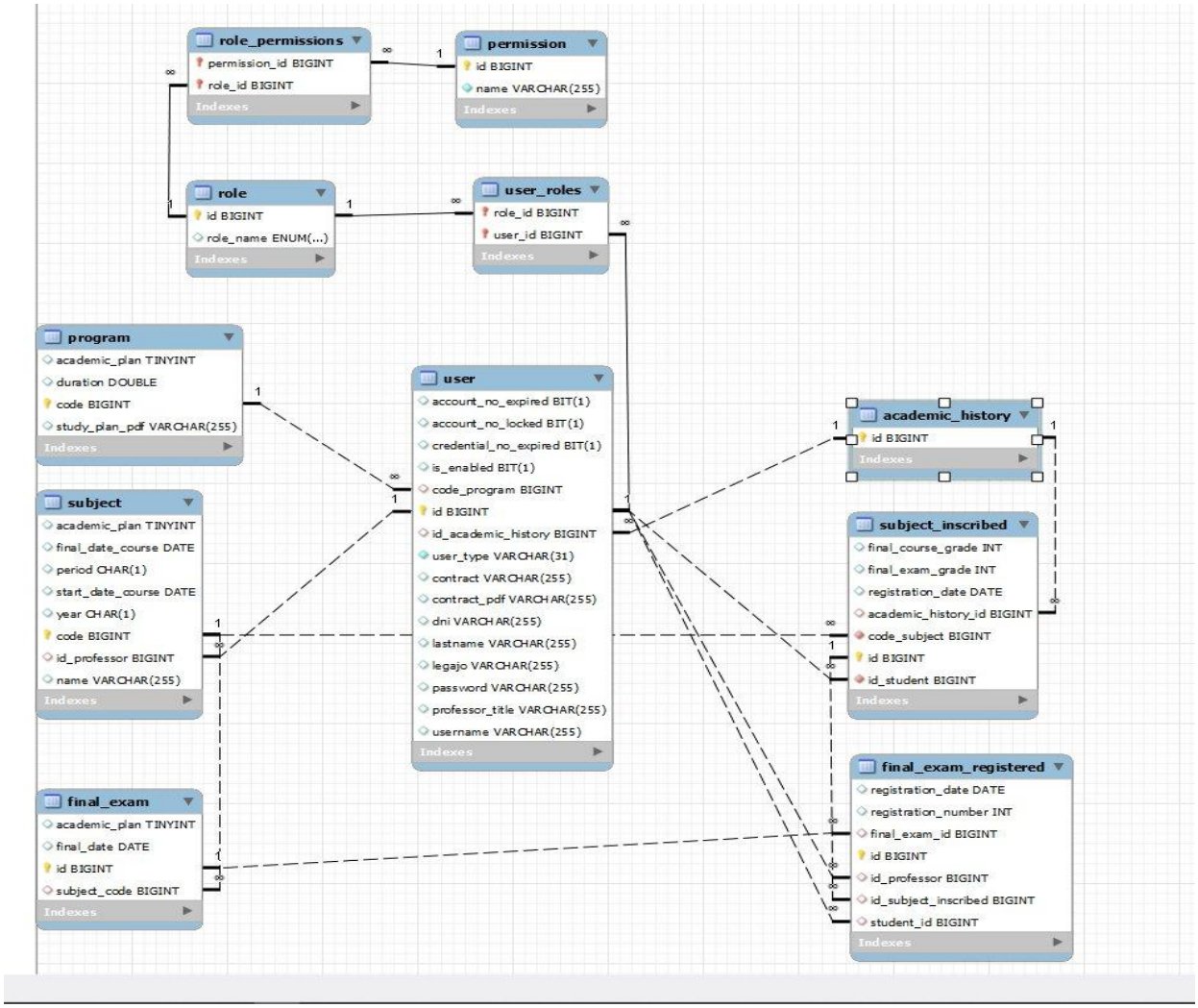

Figura 1: Diagrama Entidad-Relación del sistema Aula Virtual

4.2 Descripción de Entidades

Entidad	Descripción	Atributos Clave	Relaciones

user	Usuarios del sistema: estudiantes, docentes y administrativos	`id, username, dni, legajo, user_type, id_academic_history`	program, user_roles, academic_history
program	Programas académicos (carreras/tecnicaturas)	`code, duration, academic_plan, study_plan_pdf`	user, subject
subject	Materias que conforman un programa académico	`code, name, academic_plan, id_professor, year`	program, final_exam, subject_inscribed
final_exam	Instancias de exámenes finales por materia	`id, subject_code, final_date, academic_plan`	subject, final_exam_registered
academic_history	Trayectoria académica consolidada del estudiante	`id`	user, subject_inscribed
subject_inscribed	Inscripciones a materias con calificaciones	`id, code_subject, id_student, academic_history_id, final_course_grade, final_exam_grade`	user, subject, academic_history
final_exam_registered	Inscripciones a mesas de examen final	`id, student_id, final_exam_id, id_subject_inscribed, id_professor, registration_number`	user, final_exam, subject_inscribed
role	Roles disponibles en el sistema	`id, role_name (ENUM)`	user_roles, role_permissions
permission	Permisos granulares de acceso	`id, name`	role_permissions

4.3 Detalle de Entidades Principales

4.3.1 Entidad: user

Núcleo del sistema. Centraliza la información de todos los actores. El campo user_type diferencia entre estudiantes, docentes y administrativos. El campo id_academic_history vincula directamente al usuario con su trayectoria académica. Las credenciales se gestionan con flags booleanos (account_no_expired, account_no_locked, credential_no_expired, is_enabled) que permiten control granular del estado de la cuenta.

4.3.2 Entidad: subject_inscribed

Registra cada instancia de cursada de un estudiante en una materia. Almacena tanto la nota de cursada (final_course_grade) como la nota del examen final (final_exam_grade), permitiendo

calcular el estado académico completo del alumno. Se relaciona con academic_history para consolidar el expediente del estudiante.

4.3.3 Entidad: final_exam_registered

Gestiona la inscripción a mesas examinadoras. Vincula al estudiante (student_id), el examen (final_exam_id), la cursada previa (id_subject_inscribed) y el docente evaluador (id_professor). El registration_number provee un identificador de orden de inscripción para la mesa.

4.3.4 Sistema RBAC: role / permission / role_permissions / user_roles

Implementa un modelo de control de acceso basado en roles (Role-Based Access Control). Un usuario puede tener múltiples roles (user_roles), cada rol tiene un conjunto de permisos (role_permissions), y los permisos se evalúan en la capa de seguridad de Spring para proteger cada endpoint. El campo role_name se define como ENUM, garantizando integridad de los valores permitidos.

5. Stack Tecnológico

La selección tecnológica prioriza la robustez, la madurez del ecosistema y la compatibilidad con entornos académicos y productivos. Todos los componentes seleccionados son de código abierto o cuentan con licencias compatibles con uso educativo.

Tecnología	Versión / Tipo	Propósito
Java	JDK 17+	Lenguaje principal del backend
Spring Boot	Framework	Framework principal: REST API, seguridad, inyección de dependencias
Spring Security	Módulo	Autenticación y autorización basada en roles y permisos
JPA / Hibernate	ORM	Mapeo objeto-relacional y gestión de transacciones
MySQL	RDBMS	Motor de base de datos relacional principal
Maven	3.8+	Gestión de dependencias y ciclo de vida del proyecto
Docker	Opcional	Containerización para despliegue y entornos reproducibles
Postman	Testing	Pruebas de endpoints REST y documentación de colecciones

6. Guía de Instalación y Configuración

6.1 Requisitos del Entorno

Componente	Versión Mínima	Notas
JDK	17+	Se recomienda JDK 21 LTS para mayor soporte
MySQL Server	8.0+	Crear schema 'aula_virtual' previamente
Maven	3.8+	O usar el Maven Wrapper incluido (./mvnw)
Docker (opcional)	20.10+	Para despliegue containerizado

6.2 Pasos de Instalación

Paso 1 — Clonar el Repositorio

Clonar el repositorio del backend desde GitHub y acceder al directorio del proyecto:

```
git clone https://github.com/tu_usuario/aula-virtual-backend.git
cd aula-virtual-backend
```

Paso 2 — Configurar la Base de Datos

Crear el schema en MySQL Server antes de iniciar la aplicación:

```
CREATE DATABASE aula_virtual CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Editar el archivo src/main/resources/application.properties con las credenciales del entorno:

```
spring.datasource.url=jdbc:mysql://localhost:3306/aula_virtual
spring.datasource.username=root
spring.datasource.password=tu_password
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

Paso 3 — Ejecutar el Proyecto

Compilar y levantar el servidor con Maven:

```
mvn spring-boot:run
```

La API quedará disponible en http://localhost:8080 por defecto. Verificar el estado del servidor consultando el endpoint de health check si está configurado.

Paso 4 — Ejecución con Docker (Opcional)

Si se dispone de Docker, se puede levantar la aplicación con los siguientes comandos:

```
docker build -t aula-virtual-backend .
docker run -p 8080:8080 aula-virtual-backend
```

Para entornos con docker-compose que incluyan MySQL como servicio, consultar el archivo docker-compose.yml en la raíz del repositorio.

7. Configuración Avanzada

7.1 Perfiles de Spring

Se recomienda configurar perfiles de Spring para separar los entornos de desarrollo, testing y producción:

Perfil	Archivo de Config.	Uso
dev	`application-dev.properties`	Desarrollo local con datos de prueba y logs detallados
test	`application-test.properties`	Pruebas automáticas con base de datos H2 en memoria
prod	`application-prod.properties`	Producción con credenciales seguras y logs reducidos

7.2 Variables de Entorno Recomendadas

Para entornos productivos, se recomienda externalizar las credenciales sensibles mediante variables de entorno en lugar de hardcodearlas en el archivo de propiedades:

```
DB_URL=jdbc:mysql://host:3306/aula_virtual
DB_USERNAME=usuario
DB_PASSWORD=password_seguro
JWT_SECRET=clave_secreta_larga_y_aleatoria
JWT_EXPIRATION=86400000
```

8. Consideraciones de Diseño y Mejores Prácticas

8.1 Seguridad

- Implementar JWT (JSON Web Tokens) para la autenticación stateless de la API.
- Utilizar BCrypt para el hash de contraseñas (provisto por Spring Security).
- Aplicar validaciones de entrada en la capa de controladores con Bean Validation (@Valid, @NotNull, etc.).
- Proteger endpoints sensibles con anotaciones @PreAuthorize basadas en roles y permisos.
- Configurar CORS restrictivamente para permitir únicamente el origen del frontend autorizado.

8.2 Calidad de Código

- Seguir los principios SOLID para el diseño de clases y servicios.
- Aplicar el patrón DTO (Data Transfer Object) para separar la capa de presentación del modelo de dominio.
- Usar el patrón Repository de Spring Data JPA para abstraer el acceso a datos.
- Implementar manejo centralizado de excepciones con @ControllerAdvice y @ExceptionHandler.

8.3 Escalabilidad Futura

- Migrar a Flyway o Liquibase para gestión de migraciones de base de datos con versionado.
- Agregar Swagger/OpenAPI 3.0 para documentación automática de endpoints.
- Implementar paginación en endpoints que retornen colecciones grandes (Pageable de Spring Data).
- Considerar caché con Redis para consultas de historial académico y materias frecuentemente accedidas.
- Incorporar pruebas unitarias (JUnit 5, Mockito) y de integración para garantizar cobertura mínima del 70%.

9. Glosario

Término	Definición
API REST	Interfaz de programación basada en el protocolo HTTP, que expone recursos a través de URLs con operaciones estándar (GET, POST, PUT, DELETE).
DER	Diagrama Entidad-Relación. Representación visual del modelo de datos y sus relaciones.
RBAC	Role-Based Access Control. Modelo de control de acceso donde los permisos se asignan a roles y los roles a usuarios.
JPA	Jakarta Persistence API. Especificación de Java para el mapeo objeto-relacional (ORM).
JWT	JSON Web Token. Estándar para la transmisión segura de información entre partes como un objeto JSON firmado digitalmente.
ORM	Object-Relational Mapping. Técnica de programación que mapea objetos de la aplicación con registros de la base de datos relacional.
DTO	Data Transfer Object. Objeto que transporta datos entre capas de la aplicación, desacoplando el modelo de dominio de la API.
DDL-auto	Propiedad de Hibernate que determina si el schema de la BD se crea, valida o actualiza automáticamente al iniciar la aplicación.
Plan Académico	Conjunto de materias y requisitos que conforman la malla curricular de una carrera o tecnicatura.

10. Historial de Versiones

Versión	Fecha	Autor	Cambios
1.0.0	2025	Equipo de Desarrollo	Versión inicial del documento. Documentación completa del modelo de datos, arquitectura, stack tecnológico e instrucciones de instalación.